Local AI Assistant for Weather and Calendar

Course: Natural Language Systems, Winter 2025-2026

1 Introduction & Project Scope

The project's goal is to develop a task-oriented conversational system that can perform two tasks. First, the system will connect to a specified API to provide answers to questions about the weather in different geopolitical locations. Second, it will manage user appointments by executing Create, Read, Update, and Delete (CRUD) operations through a designated Calendar API. An essential requirement is that the system must operate locally and only interact with permitted and mentioned APIs. Additionally, the system should keep track of conversation history, allowing users to modify their previous inputs during the interaction.

To achieve this, we utilized **Rasa**, an open-source framework that combines a hybrid NLU pipeline with a probabilistic dialogue engine. Rasa was selected specifically for its explicit separation between NLU and dialogue policies, which allows for the independent evaluation of language understanding versus conversational behavior.

Finally, this report focuses on evaluating the reliability and correctness of a locally deployed task-oriented conversational system. Evaluation is centered on intent classification accuracy, entity extraction performance, and end-to-end task completion with verified side effects. Perceptual qualities such as speech naturalness and long-term conversational engagement are considered out of scope.

2 System Architecture

The system follows a modular pipeline architecture, containerized using **Docker** to ensure consistency and portability across different environments. The main pipeline consists of five distinct stages:

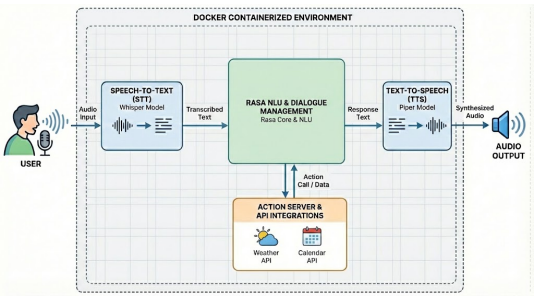


Figure 1: System Architecture Diagram *(Photo Generated by Gemini)*

- **Automatic Speech Recognition (ASR):** For this feature, we used OpenAI's *Faster-Whisper* model. It was selected for its ability to perform efficiently both on CPU and GPU hardware, and for providing high transcription accuracy for the English language, outperforming alternatives like VOSK in our testing. (ASR errors are evaluated indirectly through downstream intent and entity accuracy rather than isolated transcription metrics.)
- **Natural Language Understanding (NLU):** We utilized a hybrid pipeline where *SpaCy* handles dense embeddings and GPE extraction, *Duckling* manages temporal parsing, and the *DIETClassifier* resolves user intents and specific location entities.
- **Dialogue Management:** We employed a dual-policy approach using *Rules* for deterministic data collection (Forms) and machine-learning-based *Stories* to handle context switching and interruptions. (This separation ensures that side-effectful API calls are only triggered after explicit slot validation.)
- **Custom Actions & APIs:** We developed custom Python scripts to bridge the dialogue engine with the external Weather and Calendar APIs. These scripts enforce robust validation logic, including "Time-Travel" guards to prevent data overwrites and Regex-based cleaning. (This design allows evaluation to verify not only conversational success, but also actual backend state changes.)
- **Text-to-Speech (TTS):** Finally, we used *PiperTTS* for response synthesis, a fast local neural TTS system that delivers natural-sounding output without external cloud dependencies. The assistant uses a male voice profile ("en-US-ryan-high") to depart from the industry norm of female virtual assistants.

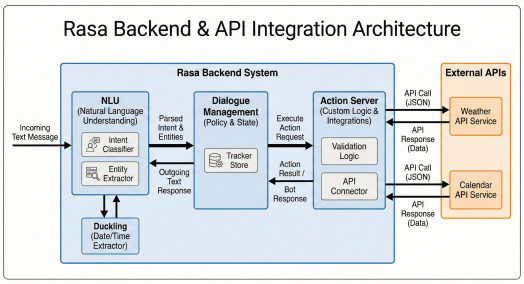


Figure 2: Rasa Architecture *(Photo Generated by Gemini)*

3 Technical Challenges and Mitigations

We encountered several architectural challenges inherent to Rasa, primarily concerning context management, entity resolution, and data miscomprehension. To resolve slot mapping issues where `from_text` mappings would erroneously fill multiple slots globally, we created strict intent conditions and scoped mappings to active forms. We also addressed entity extraction conflicts by implementing a layered strategy—delegating temporal parsing to Duckling, GPE detection to SpaCy, and specific Location (Venue) extraction to DIET —to prevent overlap. Furthermore, to mitigate policy unpredictability between deterministic and probabilistic components, we strictly sepa-

rated concerns by using Rules for rigid business logic and Stories for conversational flow. Finally, to overcome the data scarcity limitations of the supervised DIETClassifier, we utilized Generative AI for synthetic data augmentation, expanding our dataset with diverse paraphrases to ensure robust generalization.

4 Evaluation Methodology

To evaluate the system's performance, we used a two-stage evaluation approach: an intrinsic evaluation of the NLU component using rasa's internal cross-validation, and an extrinsic end-to-end blackbox test to verify system reliability and side-effects.

4.1 Intrinsic NLU Evaluation

We assessed the Natural Language Understanding (NLU) pipeline using Rasa's built-in 5-fold cross-validation technique.

From the evaluation we can see that the Rasa NLU pipeline performs well on the main task-oriented intents, with an overall accuracy of **1.0** (weighted F1 $\approx$ 1.0). The core command intents, `ask_weather` (F1 $\approx$ 1.0) and `calendar_add` (F1 $\approx$ 1.0), proved to be the most reliable. Furthermore, auxiliary intents such as `deny`, `stop`, `goodbye`, and `greet` showed a perfect performance, indicating no misclassification in the current test set.

For entity extraction, performance is high overall (micro F1 $\approx$ 0.98). The system demonstrates robust extraction for `time` (F1 $\approx$ 0.98) and `GPE` (F1 $\approx$ 0.96). Notably, `event_title` detection is now perfect (F1 $\approx$ 1.0). However, `location` extraction is slightly weaker (F1 $\approx$ 0.70) with some confusion regarding `GPE` entities, suggesting that distinguishing between generic locations and specific geopolitical entities remains a minor source of our NLU errors.

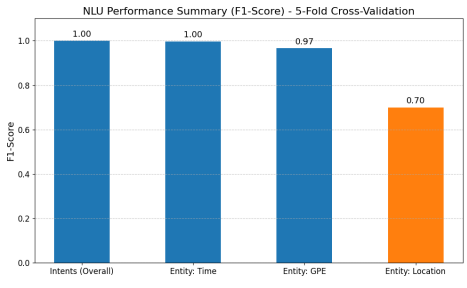


Figure 3: NLU Performance Summary

Observation: While the perfect scores on intent classification indicate high reliability within the scope of our dataset, they also suggest a potential lack of linguistic variance in the test set. The model has effectively mastered the specific command structures provided but may require further stress-testing with diverse phrasing to ensure true generalization.

4.2 Automated End-to-End System Evaluation

In addition to the NLU results, we tested the system's reliability using an **end-to-end black-box testing** method. This ensured that the core components—*Faster-Whisper*, *Rasa*, and *PiperTTS*—operate properly together. We created an automated script to evaluate the containerized system as a black box. The evaluation process enforces a four-step validation for every test case:

- **Input Injection:** The script feeds pre-recorded audio commands into the Docker container, which are converted into text transcripts using the Faster-Whisper model.
- **NLU Verification:** The script verifies if Rasa correctly identified the expected intent and extracted the necessary entities from the transcribed text.
- **Side-Effect Verification:** Crucially, for CRUD operations, the test passes *only* if the external API returns a `'200 OK'` status and the data modification is verified in the backend. A test is considered failed if the bot replies with "Done" but the API returns an error.
- **Response Validation:** Finally, the script compares the bot's text (later turned into audio via PiperTTS) response against a predefined set of success phrases to ensure the user receives the correct feedback.

Acknowledgments

We used Grammarly to improve the clarity and coherence of our sentences in this document. For our coding project, we utilized generative AI tools to generate additional training data samples, brainstorm ideas, and, most importantly, debug print statements.

References

[1] Rasa Technologies, "Rasa Open Source Documentation," [Online]. Available: [Link]. See also: "Learning Rasa Open Source," YouTube Playlist. Available: [Link].

[2] SYSTRAN, "Faster-Whisper: Faster Whisper transcription with CTranslate2," GitHub Repository. Available: [Link].

[3] OHF-Voice, "Piper1-GPL: Fast and local neural text-to-speech engine," GitHub Repository. Available: [Link].

[4] H. Chen, X. Liu, D. Yin, and J. Tang, "A Survey on Dialogue Systems: Recent Advances and New Frontiers," *SIGKDD Explorations*, vol. 19, no. 2, pp. 25-42, 2017.

[5] S. LuperFoy, D. Loehr, D. Duff, K. Miller, F. Reeder, and L. Harper, "An Architecture for Dialogue Management, Context Tracking, and Pragmatic Adaptation in Spoken Dialogue Systems," *The MITRE Corporation*, McLean, VA, USA.